NFTracker: Fine-Grained NFT Behavior Traffic Identification Over Encrypted Tunnel

Ke Ding, *Graduate Student Member, IEEE*, Xiaoyan Hu, *Member, IEEE*, Zhuozhuo Shu, Guang Cheng, *Member, IEEE*, Ruidong Li, *Senior Member, IEEE*, and Hua Wu, *Member, IEEE*

***Abstract*—Non-Fungible Tokens (NFTs) have completely changed digital ownership and the decentralized economy. However, their anonymity and encrypted communication, conducted over encrypted tunnels, pose a significant obstacle to regulating illegal activities. Despite advances in encrypted traffic analysis, fine-grained identification of NFT behaviors over encrypted tunnels faces two critical challenges: 1) inexact segmentation of continuous behavioral traffic, and 2) feature homogeneity due to encryption-induced pattern obfuscation. In this paper, we propose NFTracker, a novel framework to identify fine-grained NFT behavioral traffic over encrypted tunnels. We design a traffic segmentation method to isolate behavioral units by leveraging traffic bursts and distribution discrepancies. To combat feature homogeneity, we introduce a sliding-window-based spatio-temporal feature extraction mechanism that captures localized action fingerprints. Furthermore, we utilize a hybrid CNN-Transformer model to integrate spatial patterns and temporal dependencies for robust behavior identification. We evaluate NFTracker on real-world datasets covering five NFT behaviors (browsing, wallet login, purchasing, selling, and minting). Experimental results demonstrate that NFTracker achieves an average F1-score of 0.9212 on identifying NFT behavioral traffic, outperforming state-of-the-art methods in encrypted tunnel scenarios.**

***Index Terms*—Encrypted traffic analysis, fingerprinting, non-fungible token, spatio-temporal feature extraction.**

I. Introduction

NON-FUNGIBLE Tokens (NFTs), as an emerging paradigm of digital assets derived from blockchain technology, have revolutionized digital ownership certification mechanisms through their inherent non-fungible and indivisible characteristics. Capable of representing a wide range of digital and physical assets, NFTs have reshaped markets for art [1], collectibles [2], and gaming [3]. Serving as a conduit for value between physical and digital dimensions, NFTs drive growth in the digital economy through asset digitization and decentralized value systems.

While NFT technology has demonstrated transformative potential in many fields [4], its anonymity and decentralized nature have concurrently engendered novel cybercrime risks [5]. Such crimes often utilize encrypted tunneling technology to conceal physical locations and communication links, particularly in highly regulated regions. Encrypted tunneling techniques use encrypted encapsulation mechanisms, such as SSL/TLS protocols, to conceal sensitive operations as regular HTTPS traffic, thereby establishing highly covert network communication channels [6]. These encrypted tunnels help evade network supervision systems [7]. Their concealment ability proves particularly evident in criminal operations involving NFT cross-border money laundering, contraband trade, and illegal information dissemination.

Identifying the behaviors of NFT application traffic in encrypted tunnels is necessary to investigate fine-grained criminal activities involving NFTs. Existing research has employed traffic analysis techniques to explore Decentralized Application (DApp) fingerprinting [8], [9], but further identification of specific user operations is still required. For instance, regulators analyzing traffic patterns might detect that a corruption suspect has conducted many NFT sales activities, potentially indicating illegal asset transfers. Similarly, frequent NFT transactions by corporate holders could trigger money laundering alerts, while repetitive wallet login attempts may signal phishing attacks by malicious actors. These behavioral patterns require fine-grained traffic analysis to establish correlations between network activities and on-chain operations, enabling forensic investigations and risk mitigation within blockchain ecosystems. However, prior behavioral traffic identification research faces the subsequent two critical challenges when applied to NFT traffic over encrypted tunnels.

A. Inexact Behavioral Traffic Segmentation

Different behavioral definitions have their corresponding segmentation methods. Protocol-based definitions fail in encrypted traffic due to data-frame obfuscation. Due to transactional complexity, user-side operation mappings

Received 24 July 2025; revised 16 October 2025 and 12 December 2025; accepted 18 January 2026. Date of publication 28 January 2026; date of current version 6 February 2026. This work was supported in part by National Key Research and Development Program of China under Grant 2023YFB3106801, in part by National Natural Science Foundation of China Project under Grant 62472087, in part by the Jiangsu Province Natural Science Foundation Project under Grant BK20231413, in part by the Fundamental Research Funds for the Central Universities under Grant 2242025K30025, and in part by Postgraduate Research&Practice Innovation Program of Jiangsu Province under Grant KYCX25_0520. The associate editor coordinating the review of this article and approving it for publication was Q. Ye. *(Corresponding author: Xiaoyan Hu.)*

Ke Ding, Zhuozhuo Shu, Guang Cheng, and Hua Wu are with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: keding@seu.edu.cn; 220215193@seu.edu.cn; chengguang@seu.edu.cn; hwu@seu.edu.cn).

Xiaoyan Hu is with the School of Cyber Science and Engineering and the Engineering Research Center of Blockchain Application, Supervision and Management, Ministry of Education, Southeast University, Nanjing 211189, China, also with the Purple Mountain Laboratories for Network and Communication Security, Nanjing 211111, China, and also with Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing 211189, China (e-mail: xyhu@njnet.edu.cn).

Ruidong Li is with the Institute of Science and Engineering, Kanazawa University, Kakuma, Kanazawa 920-1192, Japan (e-mail: lrd@se.kanazawa-u.ac.jp).

Digital Object Identifier 10.1109/TCCN.2026.3658756

often struggle to align with NFT behaviors, introducing semantic ambiguities in behavior characterization. Current traffic segmentation methods face limitations. Sliding window approximation [10] introduces pseudo split points that cannot segment consecutive similar behaviors. Protocol-dependent frame alignment [11] lacks generality in encrypted environments. Both fail to precisely isolate continuous or similar behavioral units in dynamic networks.

B. NFT Traffic Feature Homogeneity

Due to the similar program interfaces of the same NFT application, its generated traffic is strongly similar. Different behaviors may also exhibit similar action logic, resulting in similar traffic patterns. Encrypted tunnels use secondary encryption and dynamic encapsulation to further homogenize traffic patterns, amplifying intra-application feature overlap and degrading fine-grained behavioral discrimination.

We propose NFTracker, a novel framework for fine-grained NFT behavior identification over encrypted tunnels to address these challenges. First, NFTracker utilizes burst and Wasserstein Distance (WD) to segment behavioral traffic, addressing inexact segmentation in continuous similar behaviors encrypted by tunnels. It computes the WD difference between the left and right windows around these candidates, using each burst's starting packet as the initial segmentation point. The random forest model infers actual segmentation points based on WD results that encrypted tunnels cannot conceal, filtering out pseudo points. Second, NFTracker employs a sliding window spatio-temporal feature extraction approach to capture unique behavioral patterns from segmented traffic, thereby solving the issue of feature homogeneity. Traverse sequences of packet length/direction and time interval through a sliding window to generate spatial and temporal feature matrices. Appropriate window size and step size ensure full coverage of the action segment, reducing noise from redundancy and thereby improving the representational ability of behavioral traffic features despite tunnel encryption. Finally, NFTracker integrates a hybrid CNN-Transformer model to learn spatio-temporal features for accurate traffic classification under encryption. It inputs spatial features into a Convolutional Neural Network (CNN) to extract local spatial patterns and uses a transformer to capture the global dependencies of temporal features. The main contributions of this paper are as follows:

- We propose a framework for fine-grained NFT behavioral traffic identification over encrypted tunnels, including behavioral traffic segmentation, spatio-temporal feature extraction, and hybrid model classification.
- We conduct a systematic analysis of the interaction patterns of five types of NFT-related traffic behaviors within encrypted tunnels, revealing the differences in traffic characteristics across different behaviors, including NFT browsing, wallet login, NFT purchasing, NFT selling, and NFT minting.
- We address inexact segmentation and feature homogeneity, two key technical challenges in behavioral traffic identification, by designing a burst-WD-based behavioral traffic segmentation method and a sliding-window-based spatio-temporal feature extraction approach. We also elaborate on their theoretical foundations for overcoming these challenges.
- We implement a prototype of NFTracker and evaluate it on two real-world datasets. Experimental results show that NFTracker achieves higher identification precision and outperforms baselines in encrypted tunnel scenarios.

II. Background and Related Work

This section provides background on the encrypted tunnel and related work on NFT behavior analysis, encrypted tunnel traffic classification, and behavioral traffic identification.

A. Encrypted Tunnel

Regulators filter encrypted tunnel traffic to preliminarily find illegal activities. An encrypted tunnel establishes an encrypted communication path between two entities to ensure data transmission security. Compared with ordinary HTTPS, the tunnel encrypts data content and hides the real destination IP, ports, and communication purposes. It decouples the observable endpoint from the actual service. Therefore, the sniffer only observes the relay address and a long-lived outer flow instead of per-request transactions. This paper focuses on application-level tunnel V2Ray [12], a mainstream implementation of the VMess [13] protocol.

When users initiate network requests through V2Ray clients, the system encrypts them according to the configured settings and forwards them to the target servers. The server decrypts the data and returns responses, which the V2Ray client subsequently decrypts and delivers to users. VMess is a stateless TCP-based encrypted transmission protocol. It means that each data transmission has no effect on other data transmissions before and after. VMess does not save session information and encrypts each packet independently during transmission, weakening the behavioral correlation between packets. Each V2Ray instance defines inbound and outbound protocols, and the protocol used by each is specified in the configuration. Preconfigured encryption keys and parameters enable direct data transmission between clients and servers without the handshake, enhancing communication efficiency. V2Ray encapsulates original application-layer traffic into encrypted and authenticated VMess frames, and then transports these encrypted VMess frames via transport mechanisms (e.g., TCP, WebSocket) over TLS as ordinary HTTPS traffic. This reframes packet boundaries and aggregates frames. Because heterogeneous applications are funneled through the same encapsulation, the packet-level features become more homogeneous across services. Essentially, VMess implements secondary encryption and encapsulation of original TCP-layer packets, which introduces latency but preserves fundamental communication patterns.

B. NFT Behavior Analysis

NFTs are blockchain tokens representing unique ownership. Their verification and transaction records rely on blockchain smart contracts, ensuring secure trading and transfer. NFTs are mainly built on Ethereum [14] smart contracts and adhere to specific standards, such as ERC-721 [15] and ERC-1155 [16].

Current NFT behavioral analysis methods primarily focus on on-chain data. Wang et al. [17] build temporal transaction graphs to analyze user behaviors and predict NFT prices. Niu et al. [18] detect wash trading, which is coordinated self-trading with rapid round-trip trades of the same NFT by a single trader or colluding wallets to inflate apparent demand and reported price and volume, using transaction graphs and strongly connected components. Huang et al. [19] characterize on-chain behavior and exchange patterns to flag wash trading and arbitrage. Cao et al. [20] simulate NFT markets with a minimum substitute model and investigate stakeholder behavior. Che et al. [21] detect malicious cryptocurrency accounts across platforms via interaction feature learning, highlighting the value of modeling account interaction patterns for illicit activity discovery.

The anonymity of blockchain prevents on-chain data from locating real-world entities. Specifically, public ledgers record wallet addresses and basic transaction metadata rather than real-world identifiers. One user can control multiple addresses or transact through custodial services that aggregate ownership, further hindering real-world entity resolution. Emerging techniques address this limitation through network traffic analysis. Shen et al. [8] proposed a DApp fingerprinting method that builds traffic interaction graphs in which each node denotes a packet by its length and edges encode intra-burst and inter-burst links. They trained a classifier based on a Graph Neural Network (GNN) to identify the DApp. Karunanayake et al. [9] model each trace as a flow node and use GNN-based node classification to fingerprint the DApp. However, fine-grained behavior identification is beyond the scope of these methods.

C. Encrypted Tunnel Traffic Classification

Several works have aimed to distinguish encrypted tunneling traffic accurately. Xue et al. [22] employ the encapsulated TLS handshake to identify obfuscated proxy traffic reliably. Moreover, Xue et al. [23] use targeted protocol features such as byte patterns, packet sizes, and server responses to identify OpenVPN traffic. Regulators further infer which websites and applications users have visited through traffic fingerprinting. Chen et al. [24] leverage the causal relationship between user requests and website responses to causally link interaction packets from the same website into causal chains. They employ contextual learning to capture dependencies among these causal chains, enabling multi-label fingerprinting attacks over encrypted tunnels. Ma et al. [25] generate encrypted traffic fingerprints by combining spatial associations through K-nearest neighbor comparisons and temporal associations via frequent traffic subsequence mining, achieving website classification under encrypted proxies. Shen et al. [26] proposed a contrastive-learning-based method for robust malicious encrypted traffic detection, showing improved resilience to traffic obfuscation. However, website-level inference alone proves inadequate to resolve the challenges addressed in this paper.

D. Behavioral Traffic Identification

Fine-grained behavioral traffic identification enables regulators to pinpoint illicit network activities effectively. It comprises two critical phases: traffic segmentation and traffic identification. Traffic segmentation encompasses three granularities: 1) Session/flow-based segmentation targeting coarse-grained behaviors (e.g., video, email). Xu et al. [27] perform explainable encrypted traffic classification through session-level path signature feature extraction. 2) Burst/time window-based segmentation targeting fine-grained operations (e.g., bitcoin transactions, application interactions). Aiolli et al. [28] defined bursts as consecutive packets within 1-second intervals combined with IP/port-based flow splitting. Ahmed et al. [29] extract features from a fixed post-invocation time window to fingerprint fine-grained voice commands. 3) Protocol transmission unit-based segmentation relying on protocol specifications. Our previous work [11] segments Ethereum node interactions using RLPx frame headers and ACK patterns. Although the classification is precise, such protocol-dependent methods lack generalizability. Subsequently, traffic identification involves feature representation and model classification. Classification approaches dynamically select traditional machine learning [28] or deep learning [10], combining statistical, sequential, and composite features according to specific scenario requirements.

Practical solutions require balancing the granularity of segmentation, efficiency, and generalizability. The identification performance depends on feature engineering and model adaptation. ActiveTracker [10] employs sliding windows to create non-overlapping segments, constructs spatio-temporal traffic matrices with spectral vectors, and applies the CNN for application behavior identification. Hou et al. [30] utilize bidirectional flow metrics, cumulative sizes, and length distributions with a random forest classifier. BehavSniffer [31] integrates statistically screened features with GCN-extracted traffic burst graph representations, achieving behavior identification via DNN-GNN joint training with linear fusion. PACKETPRINT [32] employs sequential XGBoost to quantify packet-application similarity and applies hierarchical clustering for traffic segmentation. It extracts packet structural patterns via hierarchical bag-of-words and identifies user actions with random forests.

III. Threat Model

This paper considers that regulators obtain network traffic through Internet service providers, aiming to further infer the fine-grained behavior of users in NFT exchanges, such as NFT browsing, wallet login, purchasing, selling, and minting, based on classified specific NFT application traffic over encrypted tunnels. Regulators train identification models based on labeled behavioral traffic datasets, then segment captured application traffic, and extract traffic features to identify fine-grained behaviors currently being executed by users. For example, suppose regulators already have the traffic of users conducting NFT purchase activities on the OpenSea exchange through V2Ray. In that case, they can infer whether users secretly purchase on the OpenSea exchange over V2Ray through real-time traffic identification.

Specifically, we assume a regulator's sniffer between encrypted tunnel clients and servers. Figure 1 shows a typical NFT fine-grained behavior identification scenario. Original traffic is generated between NFT application clients and

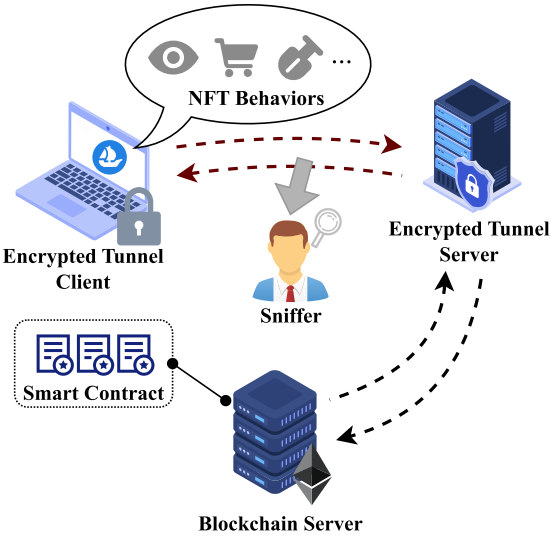


Fig. 1. A typical scenario of NFTracker.

blockchain servers equipped with smart contracts. Similar to prior website fingerprinting works [25], [33], the sniffer cannot actively modify packets or probe target servers, nor decrypt the encapsulated application-layer payload within encrypted tunnels to obtain NFT application addresses, port numbers, or plaintext payloads. In fact, the regulator is already able to use website fingerprinting technology [25], [33] to identify specific applications. However, the regulator's goal is to further passively infer which specific behavior the NFT traffic in the tunnel corresponds to.

IV. System Design

This section presents NFTracker, a method for fine-grained behavioral traffic identification of NFT transactions over encrypted tunnels. NFTracker addresses the two core challenges of inexact segmentation and feature homogeneity by integrating burst-WD segmentation, sliding windows, and a hybrid CNN-Transformer model. Figure 2 shows the framework of NFTracker. It comprises behavioral traffic segmentation, spatio-temporal feature extraction, and hybrid model classification. It achieves high-precision identification of NFT fine-grained behaviors even over encrypted tunnels.

Behavioral Traffic Segmentation: NFTracker introduces a behavioral traffic segmentation algorithm based on bursts and WD. Initial segmentation points are identified by detecting burst starts in bidirectional session flows. The WD is then employed to quantify the distributional differences in packet lengths and directions between the left and right windows around each candidate point. A random forest classifier filters out pseudo-segmentation points, ensuring high segmentation accuracy. NFTracker's segmentation mechanism bypasses protocol encryption from encrypted tunnels by exploiting intrinsic request-response packet sequences.

Spatio-Temporal Feature Extraction: NFTracker extracts spatio-temporal features using a sliding window approach. Preprocessed packet length and direction sequences are transformed into spatial feature matrices, while inter-packet time intervals are converted into temporal vectors. Overlapping sliding windows capture localized traffic patterns associated with specific actions [34], preserving the integrity of unique behavioral signatures and enhancing discriminative feature representation.

Hybrid Model Classification: NFTracker employs a hybrid CNN-Transformer model to fuse spatial and temporal features. A CNN processes the spatial feature matrix to capture local spatial patterns. At the same time, the temporal feature matrix is fed into a Transformer encoder to model long-range dependencies through multi-head attention mechanisms. The outputs are concatenated and fed into fully connected layers with softmax activation to generate the final behavioral traffic classification probabilities, achieving robust identification of five NFT behaviors: browsing, wallet login, purchasing, selling, and minting.

A. Behavioral Traffic Segmentation

Behavioral traffic segmentation aims to precisely partition NFT application traffic into packet segments corresponding to individual user behaviors. Specifically, the process involves granularly dividing a bidirectional session flow into packet subsequences. These session flows contain multiple behavioral activities. It ensures explicit correspondence between each segment and its specific behavior. We propose a machine learning-based behavioral traffic segmentation method that utilizes packet direction and the WD of length distributions as parameters, with bursts as initial segmentation points.

1) Behavior Definitions: Behavioral definitions dictate the granularity of traffic segmentation. Based on user interactions and functional modules within NFT exchanges, mainstream NFT exchange behaviors are categorized into NFT browsing, wallet login, NFT purchasing, NFT selling, and NFT minting. These classifications enable regulators to differentiate benign activities (e.g., browsing, login) from potentially risky operations (e.g., abnormally high-frequency purchases, unauthorized minting). Specifically, the following definitions are provided in this paper.

Definition 1 (Behavior): A behavior is defined as a goal-driven process initiated by a user and composed of a series of actions.

For example, browsing an NFT collection to view item metadata and purchasing a selected NFT each qualify as user-initiated behaviors realized through multiple actions.

Definition 2 (Action): An action is a low-level sub-operation within a behavior, manifested as various functions triggered during the process.

Within the NFT browsing behavior, the user request to load a page resource and the exchange response to return item metadata constitute two actions. In addition to requests and responses between users and exchanges, the NFT purchasing behavior consists of the user confirmation and wallet/exchange verification actions.

Definition 3 (Session Flow): A session flow is a bidirectional sequence of packets exchanged between an encrypted tunnel client and an encrypted tunnel server.

The segmentation approach builds upon a critical observation that distinct behavioral traffic streams merge into a single bidirectional session flow after V2Ray processing. Temporally, we observe alignment between bidirectional session durations

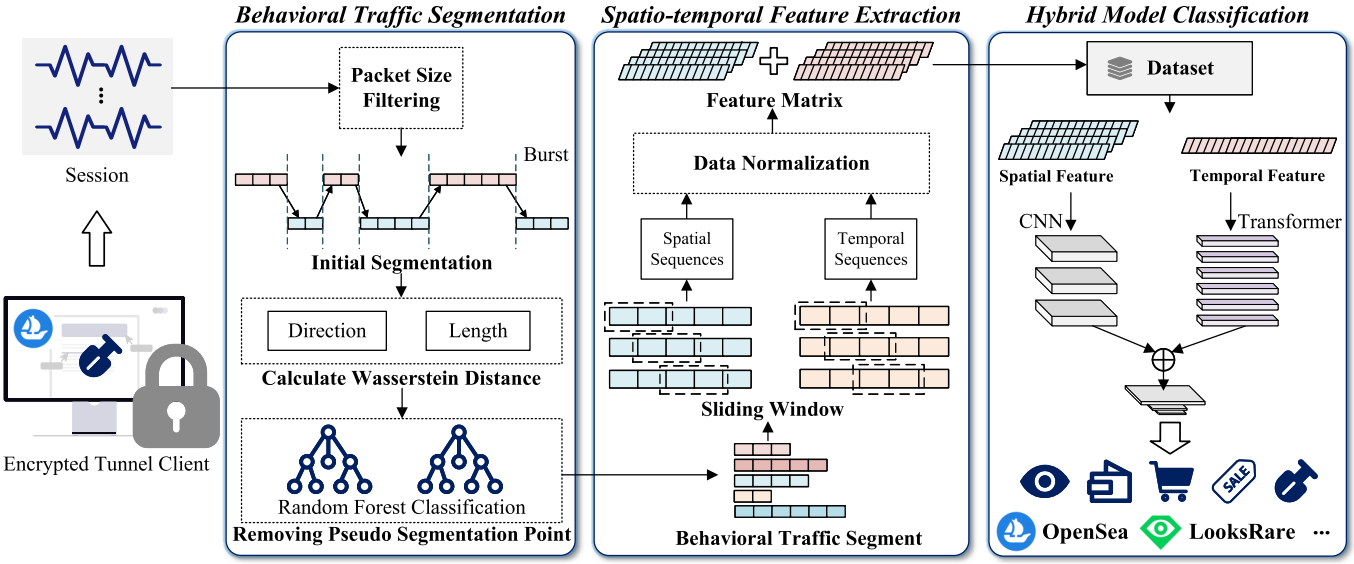


Fig. 2. The framework of NFTracker.

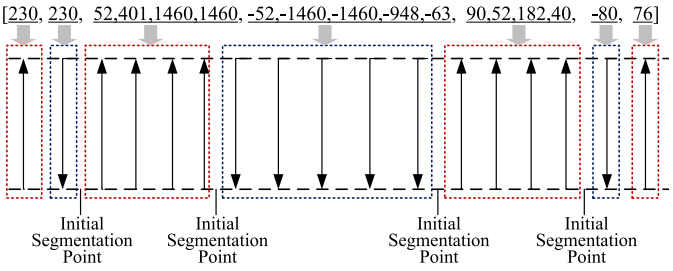


Fig. 3. An example of the burst.

and cumulative behavioral durations. For content analysis, we utilize system variables to log TLS handshake key information and then decrypt the corresponding traffic using Wireshark. Decrypted traces reveal operational functions corresponding to each behavior within unified flows, confirming that heterogeneous behavioral traffic aggregates into single bidirectional sessions post-V2Ray.

2) Traffic Segmentation: The segmentation of behavioral traffic involves three steps: calculating initial segmentation points, computing the WD between left and right windows, and random forest classification. First, burst start positions are selected as initial segmentation points. To ensure extracted features reflect application-layer characteristics, packets with ACK flags and zero TCP payload lengths are discarded during preprocessing. As shown in Figure 3, a burst is defined as consecutive packets sharing the same direction within a specific timeframe, forming a coherent packet sequence. Under VMess encryption, packet payloads are obscured, yet directional consistency in burst sequences (e.g., client-to-server requests vs. server-to-client responses) preserves behavioral boundaries. According to NFT application interaction logic, user-initiated requests precede application responses. User requests generate server-bound data bursts, while server responses form client-directed bursts. Thus, behavioral segments inherently start with burst initiation. However, complex behaviors like NFT selling may involve multiple bursts (e.g., NFT listing, wallet authorization, transaction confirmation).

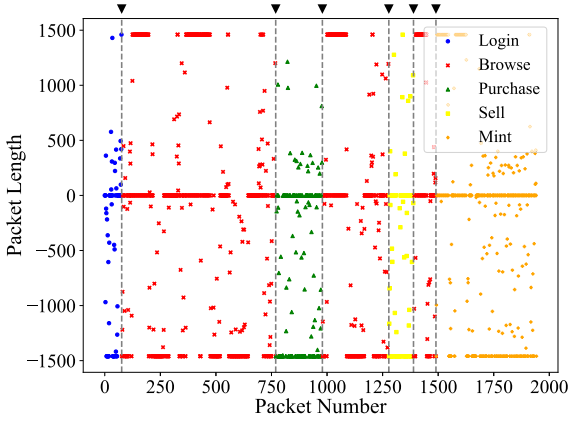


Fig. 4. Five behavioral patterns within a single session flow.

To filter initial segment points, NFTracker uses the WD between packet distributions as the classification basis for the random forest model. Behavioral distinctions manifest as differences in packet length and directional distribution. Figure 4 illustrates five behavioral patterns within a single session flow, plotting packet directions (positive/negative signs) against lengths. The packet distribution of NFT purchase behaviors is mainly concentrated in the negative direction, where the packets with lengths not reaching the MTU exhibit a relatively uniform distribution characteristic. Meanwhile, wallet login exhibits sparse distribution with low MTU-proportion packets. In contrast, NFT browsing presents a balanced directional distribution with a high proportion of MTU packets. NFT selling reveals a negative-direction concentration. Additionally, NFT minting shows packets clustered in the 0–500 range in the positive direction. We calculate probability distributions of packet lengths/directions within symmetric windows centered at segmentation points.

The WD quantifies distribution discrepancies between left (p) and right (q) windows:

$$WD(p,q) = \inf_{\gamma \in \Pi(p,q)} \int_{X \times Y} d(x,y) d\gamma(x,y). \tag{1}$$

WD is a measure of the difference between two probability distributions, and its core idea is the minimum "transportation cost" required to transform one distribution into another. Among them, p and q correspond to two probability distributions, $d(x, y)$ is the distance between any two points x, y in the space $X \times Y$, γ is the joint distribution, and its marginal distributions are p and q respectively. $\Pi(p, q)$ is the set of all possible joint distributions with marginal distributions p and q. inf means finding the plan that minimizes the total "transportation cost" among all possible transportation plans γ. The WD ranges from 0 to positive infinity. The larger it is, the greater the difference between the two probability distributions. WD is finite and stable on sparse histograms and remains insensitive to zeros and support mismatch. It yields smoother within-behavior distances and sharper boundary changes in encrypted traffic, improving segment point classification. We compute the WDs between the left and right windows for the packet-length and packet-direction sequences at each candidate segmentation point. These distances are used as features for a random forest classifier that infers whether the point is an actual boundary or a pseudo point.

Algorithm 1 Behavioral Traffic Segmentation Point Identification

Input: Original behavioral traffic S, Interval size r, Segmentation point identification model RF_model
Output: Actual behavior segmentation point set $split\_set$

Initialization $initial\_set, split\_set$
Set $initial\_set = Extract\_Burst(S)$
for each $initial\_point$ in $initial\_set$ **do**
  Set $right = S[initial\_point : initial\_point + r]$
  Set $left = S[initial\_point - r : initial\_point]$
  $Length\_seq\_LR, Direction\_seq\_LR = Extract\_Sequence(Right, Left)$
  for seq in $[Length\_seq\_LR, Direction\_seq\_LR]$ **do**
    Compute $pd\_left, pd\_right = Probability\_Distribution(seq)$
    Compute $wd\_value = Wasserstein\_Distance(pd\_left, pd\_right)$
    Add wd_value to wd_list
  end for
  Get $result = RF\_model(wd\_list)$
  if $result == true\_point$ **then**
    Add $initial\_point$ to $split\_set$
  end if
end for

The identification algorithm for behavior flow segmentation points is shown in Algorithm 1. It first initializes an initial set of candidate points and a split set of confirmed actual points. Candidates are seeded by the start positions of bursts extracted from bidirectional session flows. For each candidate, the algorithm collects traffic intervals on both sides with a preset region size and extracts packet length and direction sequences. These sequences are then utilized to compute probability distributions for both packet lengths and directions. It computes WDs between the bilateral distributions for both modalities and feeds these distances to a classifier. Candidates predicted as true boundaries are added to the final split set S_F.

B. Spatio-Temporal Feature Extraction

NFTracker employs a sliding-window method to capture traffic patterns corresponding to distinct actions within behavioral traffic. It subsequently extracts spatial features from packet sequences and augments them with temporal features derived from the time interval, resulting in comprehensive spatio-temporal feature representations.

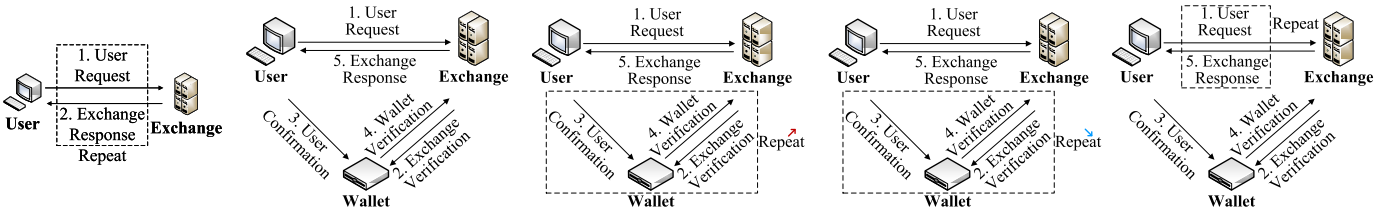
1) Analysis of Behavioral Patterns: We conduct a detailed analysis of behavioral patterns in NFT exchange platforms to help identify discriminative features that effectively characterize user behavior. According to our threat model, this paper focuses exclusively on user-exchange interaction traffic, excluding modifications to the Ethereum blockchain data via smart contracts. We classify NFT exchange behaviors into five categories: NFT browsing, wallet login, NFT purchasing, NFT selling, and NFT minting, with interactions abstracted across three core entities: users, wallets, and exchanges.

a) NFT Browsing: It typically involves only two entities (user and exchange), as shown in Figure 5 (a). The user initiates resource requests (e.g., images, descriptions) to the exchange, which responds with the requested content. These browsing sessions consist of multiple sequential queries. To visualize traffic dynamics, we convert packet lengths into graphical representations in Figure 6 (a). During request phases, user-originated packets exhibit fluctuating variations in short sequences, while response phases are dominated by consecutive MTU-sized packets from the exchange, forming distinct plateau patterns. The request-response cycle is periodic and separated by idle gaps before the next query.

b) Wallet Login: It engages all three entities, as shown in Figure 5 (b). The user sends a login request to the exchange, which initiates verification with the wallet. Upon receiving verification instructions, users confirm authentication through the wallet interface, which subsequently returns authorization tokens to the exchange. After verification, the exchange delivers account resources to the user. As depicted in Figure 6 (b), the traffic follows three phases: initial login requests from the user, verification confirmation traffic between the wallet and exchange, and resource-loading plateaus characterized by sustained MTU-sized packets.

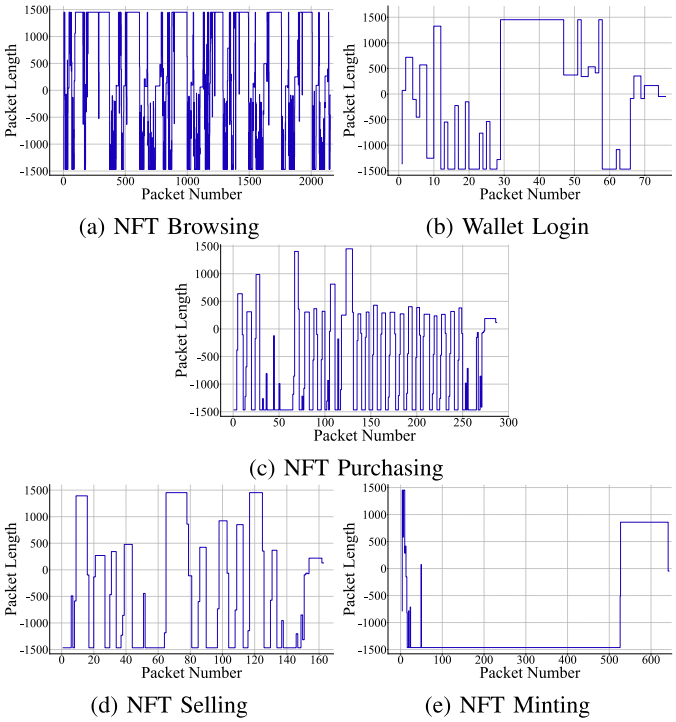
c) NFT Purchasing: It involves all three entities, as shown in Figure 5 (c). The user requests to purchase the target NFT from the exchange with transaction parameters, and the exchange then notifies the wallet. The subsequent phase involves repeated signature validations between the user, exchange, and wallet, culminating in on-chain confirmation of the transaction and the exchange's final response to the user. Figure 6 (c) shows that the repetitive validation process generates complex packet length variations, distinguishing it from simpler behaviors through multi-stage directional shifts and intermittent high-volume traffic bursts.

d) NFT Selling: As shown in Figure 5 (d), the NFT selling shares structural similarities with purchasing, but diverges in action. Key distinctions include metadata during listing NFTs and reduced wallet interaction frequency. Figure 6 (d)



(a) NFT Browsing (b) Wallet Login (c) NFT Purchasing (d) NFT Selling (e) NFT Minting

Fig. 5. The workflow of each NFT behavior.



(a) NFT Browsing (b) Wallet Login (c) NFT Purchasing (d) NFT Selling (e) NFT Minting

Fig. 6. The packet length distribution of each behavior.

shows that the authorization phase involves fewer iterative validations, reflected in fewer packet direction changes than purchasing behavior.

e) NFT Minting: It follows a multi-phase workflow shown in Figure 5 (e). Users iteratively upload content and confirm metadata with the exchange, then request wallet authorization for gas fees. In Ethereum, gas is the unit that measures computation and storage resources [14]. Users pay a gas fee to have a transaction executed and included in a block. The sender can set a fee cap, and the offered fee affects transaction confirmation time. Figure 6 (e) shows that the behavior exhibits unique traffic signatures that sustained MTU-level upload bursts during content submission, followed by wallet interactions for authorization. The completion phase shows abrupt directional shifts as the exchange finalizes minting.

While packet lengths and directional sequences provide initial discriminative features across behaviors, direct input of full sequences risks feature degradation due to inter-class similarities (e.g., identical wallet verification patterns in purchasing and selling). To address the limitation, we implement a sliding window mechanism to generate localized action fingerprints.

2) Sliding Window Based Feature Extraction: We propose a feature extraction method based on the sliding window to better capture the effective information in the sequences of packet lengths and directions. It enhances feature distinctiveness by isolating characteristic segments while preserving spatio-temporal relationships within behavioral traffic. Notably, it addresses VMess encryption-induced traffic homogeneity by focusing on relative spatial and temporal variations.

The length and direction sequence of n data packets is expressed as $Seq = \{p_0, p_1, p_2, \ldots, p_{n-1}\}$. The value p corresponds to the TCP payload length with its sign indicating the packet direction, where positive and negative values represent forward and reverse transmissions, respectively. To construct the spatial feature matrix $\mathbf{F}_s$, the packet length-direction sequence Seq is first extracted from the behavioral segment set S_F, followed by sliding-window-based feature extraction. A window of size W iteratively slides over Seq with step-size s, capturing W-length subsequences $\{p_j, p_{j+1}, \ldots, p_{j+W-1}\}$, where $j = (i-1) \times s$ for the i-th row, as row vectors in $\mathbf{F}_s \in \mathbb{R}^{N \times W}$. The window slides N times, incrementing s steps per iteration until N features populate the matrix rows, thereby converting the temporal sequence into an $N \times W$ spatial representation through overlapping window sampling.

To augment behavioral characterization, we integrate inter-packet arrival intervals as temporal descriptors alongside spatial features, where the timestamp sequence of n data packets is $\{t_0, t_1, t_2, \ldots, t_{n-1}\}$. Similarly, time-interval subsequences $\{t_{j+1} - t_j, t_{j+2} - t_{j+1}, \ldots, t_{j+W} - t_{j+W-1}\}$ are captured to construct the temporal feature matrix $\mathbf{F}_t \in \mathbb{R}^{N \times W}$. Next, we normalize the obtained spatio-temporal features.

A single behavior spans several actions implemented by the exchange, and the packets for each action often cluster into compact segments, forming distinct traffic patterns. However, partitioning the entire sequence into fixed-length non-overlapping segments risks fragmenting these unique patterns across adjacent segments due to their inherent relevance. Simply increasing the window size does not resolve this issue. With a fixed sequence length, larger windows produce fewer segments, which lowers analytical granularity, reduces the number of detectable actions, and drifts away from actual traffic characteristics.

To preserve pattern integrity, we employ overlapping windows to extract localized action fingerprints. It ensures that other windows can probabilistically recover their complete structure even if one window disrupts a specific pattern. The sliding window segments long sequences into multiple short windows, each potentially corresponding to specific operational phases, such as request, verification, and payment in NFT purchasing. It enables the model to focus on localized critical information and identify distinct behavioral patterns. Moreover, features extracted from sequential packet sequences

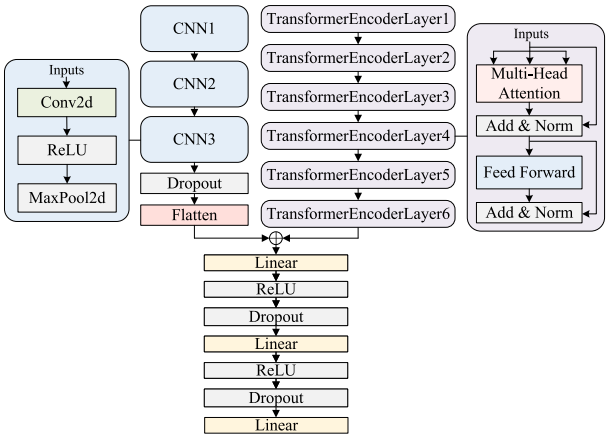


Fig. 7. Hybrid CNN-transformer network.

rather than fixed-timeframe statistical aggregations effectively mitigate noise induced by network fluctuations.

C. Hybrid Model Classification

We propose a hybrid CNN-Transformer architecture to solve the issue of high intra-class similarity in encrypted NFT traffic and enhance fine-grained behavioral identification. The model integrates spatial and temporal feature representations through parallel branches, leveraging the strengths of CNN for localized pattern extraction and Transformer for modeling long-range dependencies.

NFTracker utilizes convolutional layers to identify localized spatial patterns, followed by pooling layers for dimensionality reduction and hierarchical feature extraction. The CNN architecture effectively captures row-wise behavioral patterns within feature matrices, where each row corresponds to specific action characteristics, enabling comprehensive behavioral representation learning. For the extracted temporal feature vectors, the Transformer network utilizes its self-attention mechanism to model the long-range dependencies and temporal correlations across sequential actions. Accordingly, NFTracker adopts a parallel hybrid model integrating CNN and Transformer networks as the classification backbone. As shown in Figure 7, the left branch processes spatial features through the CNN framework, while the right branch processes temporal features via the Transformer architecture, enabling synergistic spatio-temporal behavioral characterization.

The left network consists of three convolutional blocks, a dropout layer, and a flatten layer. Each convolutional block is composed of a Conv2D layer, a ReLU activation function, and a max-Pooling layer. The CNN processes $\mathbf{F}_s$. The spatial embedding $\mathbf{E}_s$ is obtained as:

$$\mathbf{E}_s = \text{Flatten}\left(\mathcal{C}(\mathbf{F}_s)\right) \in \mathbb{R}^{D_s}, \tag{2}$$

where $\mathcal{C}(\cdot)$ represents the CNN operations, and D_s is the flattened dimension.

On the right network, the temporal sequence $\mathbf{F}_t$ is encoded by 6 Transformer encoder layers with multi-head attention and feed-forward networks. The temporal embedding $\mathbf{E}_t$ is computed as:

$$\mathbf{E}_t = \mathcal{T}(\mathbf{F}_t) \in \mathbb{R}^{D_t}, \tag{3}$$

TABLE I
HYPERPARAMETER SETTINGS OF NFTRACKER

Hyperparameter	Search Range	Value
Optimizer	[SGD, SGD+Momentum, Adam, RMSProp]	Adam
Training Epochs	[10...50]	50
Learning Rate	[0.001 ... 0.01]	0.001
Number of Heads	[1...8]	5
Activation Functions	[ReLU, Tanh, ELU]	ReLU
Batch Size	[16 ... 128]	32

where $\mathcal{T}(\cdot)$ denotes the Transformer encoder, and D_t is its output dimension. The design of the multi-head attention mechanism enables the model to simultaneously learn information from different representation subspaces, enhancing the model's representation ability.

The outputs of the two branches are concatenated and then fed into the fully connected layer to complete the final classification. The fused representation combines spatial and temporal embeddings:

$$\mathbf{E}_{\text{fused}} = \text{Concat}(\mathbf{E}_s, \mathbf{E}_t) \in \mathbb{R}^{D_s+D_t}. \tag{4}$$

It is passed through three linear layers to compute classification scores:

$$\hat{\mathbf{y}} = \text{Softmax}\left(W_3\,\phi\left(W_2\,\phi\left(W_1\mathbf{E}_{\text{fused}} + b_1\right) + b_2\right) + b_3\right), \tag{5}$$

where W_i and b_i are learnable parameters, and ϕ denotes the ReLU activation. Cross-entropy is used as the loss function. Finally, NFTracker utilizes the hybrid CNN-Transformer model to learn spatio-temporal features, effectively identifying the traffic of NFT application behaviors over the encrypted tunnel.

V. EVALUATION

In this section, we evaluate the performance of our behavior segmentation and sliding-window-based feature extraction approach. We also compare our work with state-of-the-art methods for behavioral traffic identification.

A. Experimental Setup

The NFTracker prototype is implemented based on Python 3.11, primarily using dpkt, PyTorch, and Selenium libraries. Our device operating system is Ubuntu 20.04 LTS, with an Intel (R) Core (TM) i9-12900KF CPU, 32GB of memory, and an NVIDIA GeForce RTX 4090 GPU. We select the optimal hyperparameter combination within the hyperparameter search space to implement NFTracker. The search space and the hyperparameter values are shown in Table I.

We first conduct experiments to verify the performance of the behavioral traffic segmentation approach under different window radii. Then, we evaluate the spatio-temporal feature extraction based on the sliding window approach and verify the optimal sliding window parameters. Finally, we compare NFTracker with state-of-the-art methods to verify its performance in identifying NFT behavioral traffic over the encrypted tunnel.

We evaluate our proposed method using the following indicators: Accuracy = $\frac{TP+TN}{TP+FP+TN+FN}$; Precision = $\frac{TP}{TP+FP}$;

TABLE II
DATASET DESCRIPTION

Dataset	Type	Instances
D_1	Behavior Segment Point	2116
D_2	NFT Browsing Traffic over V2Ray	2000
	Wallet Login Traffic over V2Ray	2000
	NFT Purchasing Traffic over V2Ray	2000
	NFT Selling Traffic over V2Ray	2000
	NFT Minting Traffic over V2Ray	2000

Recall = $\frac{TP}{TP+FN}$; F1-score = $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision}+\text{Recall}}$. Based on the prediction results, we calculate these indicators by the number of true positives (TPs), false positives (FPs), true negatives (TNs), and false negatives (FNs).

B. Dataset

Our dataset includes five fine-grained NFT application behaviors, namely NFT browsing, wallet login, NFT purchasing, NFT selling, and NFT minting. During traffic collection, the V2Ray client is configured with TCP as the transport protocol, encrypted using the VMess protocol, while multiplexing is disabled. The network interface card is configured with TCP Segmentation Offload disabled. Tshark is used to capture traffic from the specified network interface with promiscuous mode deactivated. To mitigate experimental deviations caused by single-device collection, we deploy multiple geographically distributed servers for cross-regional traffic acquisition, thereby reducing the impact of network environment heterogeneity. The traffic collection process utilizes the Selenium library and manual operations to simulate diverse behaviors. By analyzing page resource loading patterns, we randomly select corresponding NFT resources to execute specific fine-grained behaviors. For NFT minting behaviors, a backup resource library containing images, audio files, and other media types is established, with random selections made during minting executions. To ensure precise capture of fine-grained behavioral traffic, we measure the average time difference between packet generation and behavioral execution as the capture latency. Then, we implement delayed Tshark initialization synchronized with behavioral triggers.

Dataset D_1 contains 81 pcap files capturing complex composite behaviors, comprising 2,116 candidate segmentation point instances. Each candidate is labeled as actual or pseudo according to the behavior boundaries. D_1 is used exclusively to train and validate the segmentation random forest. Dataset D_2 specifically targets mainstream NFT marketplaces like OpenSea and LooksRare, collecting 2,000 bidirectional session flows for each of the five fine-grained behaviors. The dataset structure is detailed in Table II, which is partitioned into training, validation, and testing subsets in an 8:1:1 ratio.

C. Baselines for Comparison

To the best of our knowledge, NFTracker is the first work to identify NFT behavioral traffic over the encrypted tunnel. To make a comprehensive comparison, we select five state-of-the-art methods described in Section II, namely GraphDApp [8], ActiveTracker [10], PACKETPRINT [32], Hou et al. [30], and BehavSniffer [31]. Among them, GraphDApp [8] is a DApp traffic classification method, and the other four are behavioral traffic identification methods. Only ActiveTracker [10] and PACKETPRINT [32] have the ability to segment behavioral traffic. All baselines are trained and tested in the same experimental environment as NFTracker. We configured these methods according to their publicly available parameters. To ensure a fair comparison, we fine-tuned these baselines to guarantee the consistency of input data.

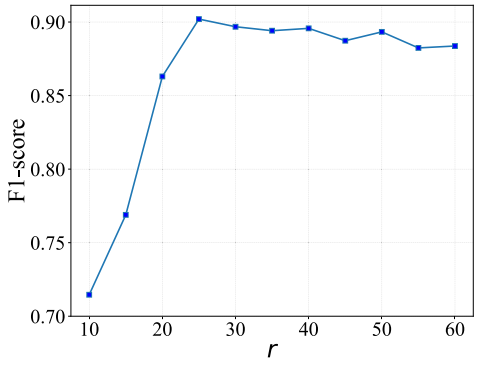


Fig. 8. Segmentation performance of different window radii.

D. Evaluation of Behavioral Traffic Segmentation

This section evaluates the effectiveness of the proposed burst-WD segmentation method, including parameter optimization and comparison with baselines.

1) Segmentation Parameter Selection: We evaluate the impact of window radius r on segmentation accuracy and search for the optimal r by training a random forest classifier across varying r values.

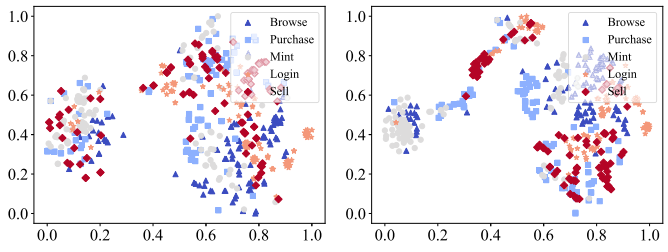
Experimental Setting: The segmentation accuracy of traffic behavior is significantly affected by the window radius r (the size of the intervals on both the left and right sides). To determine the optimal parameter, based on the packet scale of typical NFT behavioral traffic, we set the search range of r as $[10, 60]$ with a step size of $\Delta r = 5$. For each r, we extracted packet length and direction distributions from symmetric left-right windows and trained a random forest classifier.

Result: As shown in Figure 8, the experimental results demonstrate a clear correlation between window radius and segmentation performance. When $r < 15$, limited window coverage fails to capture sufficient statistical characteristics of packet distributions, resulting in a lower F1-score. As r increases to 25, the WD effectively amplifies distribution discrepancies between adjacent windows, achieving a peak F1-score with both precision and recall exceeding 0.9. The optimal balance minimizes segmentation errors while preserving discriminative distribution patterns. Beyond $r = 25$, F1-scores exhibit marginal fluctuations due to redundant noise from oversized windows diluting local feature distinctiveness. Consequently, $r = 25$ is selected as the optimal parameter.

2) Comparison With Existing Methods: Accurate segmentation of behavioral traffic is a prerequisite for ensuring accurate behavior identification. Next, we compare our approach with existing methods to evaluate the performance of NFTracker's behavioral traffic segmentation module.

TABLE III
COMPARISON OF BEHAVIORAL TRAFFIC SEGMENTATION METHODS

Method	Precision	Recall	F1-score
NFTracker	0.9361	0.8702	0.9020
ActiveTracker [10]	0.8939	0.7491	0.8151
PACKETPRINT [32]	0.8327	0.9225	0.8753



(a) Original Sequence (b) Sliding Window

Fig. 9. The t-SNE visualization of features from different methods.

Experimental Setting: We select ActiveTracker [10] and PACKETPRINT [32], which can segment behavioral traffic, as baselines to compare the performance of traffic segmentation on the D_1. We considered the three packet ranges before and after the segmentation point to be the correct segmentation.

Result: As shown in Table III, NFTracker outperforms baseline methods with an F1-score of 0.902. The advantage stems from two key innovations. The burst-WD segmentation leverages directional burst boundaries and WD to quantify distribution shifts, effectively isolating behavioral units in V2Ray traffic. Further, the random forest classifier filters pseudo-segmentation points by learning from WD-based distribution discrepancies. In contrast, ActiveTracker's sliding window method introduces missing segmentation. Additionally, PACKETPRINT's hierarchical clustering struggles with the homogeneity of encrypted NFT traffic. These results validate the NFTracker's capability to address inexact segmentation in NFT traffic over the encrypted tunnel.

E. Evaluation of Sliding-Window-Based Feature Extraction

In this experiment, we evaluate the effectiveness of sliding windows and search for the optimal parameters.

1) Impact of Sliding-Window-Based Feature Extraction on Feature Representation: NFTracker focuses on the atomic action features of behavior through sliding windows, resisting global feature homogenization caused by encrypted tunnels and similar behavior actions. First, we evaluate the effectiveness of the sliding window approach.

Experimental Setting: To evaluate the effectiveness of the feature extraction using the sliding window in NFTracker, we employ t-Distributed Stochastic Neighbor Embedding (t-SNE) to perform dimensionality reduction and visualization on the original feature sequence and the features extracted by the sliding window under the same traffic segmentation. For preliminary evaluation, a moderate value is adopted, with the window size set as $W = 30$ and the sliding step size as $s = 4$.

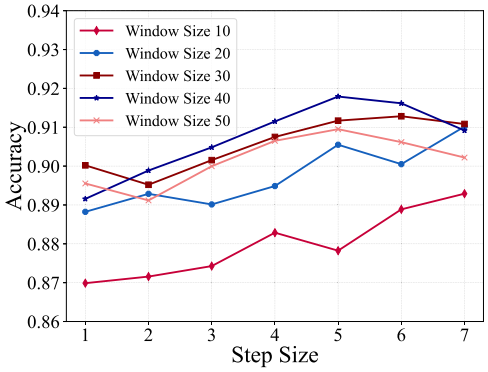


Fig. 10. Accuracy of different parameter combinations.

Result: As shown in Figure 9, the features processed by the sliding window exhibit significant intra-class aggregation and inter-class separation in the two-dimensional mapping space. This distinct clustering pattern contrasts sharply with the original features, which show substantial overlap across behavioral classes due to encryption and similar behavior. However, the windowed features can preserve localized action fingerprints while filtering out noise from redundant packets. It indicates that the windowed features can effectively enhance the model's representation ability of behavioral traffic, thus supporting a higher classification accuracy.

2) Sliding Window Parameter Selection: The sliding window parameters directly affect the model's fine-grained representation capability by regulating the spatiotemporal granularity of features. Reasonable selection of the window size W and the sliding step size s is crucial for balancing feature integrity, noise suppression, and temporal modeling.

Experimental Setting: In order to verify the optimal combination of window parameters, based on the same traffic segmentation, the experiment traverses the parameter space of the sliding window on NFTracker to classify the behavioral traffic. Among them, the window size $W \in [10, 50]$, with a step size of $\Delta W = 10$, and the sliding step size $s \in [1, 7]$, with a step size of $\Delta s = 1$. By evaluating the classification accuracy of different combinations of (W, s), the optimal parameters are screened to balance the requirements of feature coverage and noise suppression.

Result: Figure 10 shows that the combination of the window size and the sliding step size significantly impacts the identification accuracy. As W increases from 10 to 50, the accuracy first improves and then fluctuates, with $W = 40$ performing the best. It covers the complete key action segments in NFT behaviors, such as the uploading stage of NFT minting or the verification process of wallet login, thereby enhancing the feature expression ability. A smaller window misses cross-packet dependencies, limiting the integrity of feature capture. When $W = 50$, the overall accuracy shows a downward trend. The reason is that an overly large window may contain the mixed features of multiple actions, introducing redundant noise and blurring discriminative patterns. Then, the step $s = 5$ provides moderate overlap, so transitional edges are observed by consecutive windows. Smaller step sizes increase overlap between adjacent windows and inflate feature redundancy, whereas larger step sizes shrink feature coverage and risk

TABLE IV
PERFORMANCE COMPARISON OF NFTRACKER AND BASELINES

Method	Precision	Recall	F1-score
NFTracker	**0.9245**	**0.9179**	**0.9212**
NFTracker-C	0.9182	0.9054	0.9118
NFTracker-T	0.8925	0.8907	0.8916
GraphDApp [8]	0.8213	0.7718	0.7958
Hou *et al.* [30]	0.7624	0.7550	0.7587
BehavSniffer [31]	0.8512	0.8784	0.8646
ActiveTracker [10]	0.8285	0.8301	0.8293
PACKETPRINT [32]	0.8873	0.8732	0.8802

missing key temporal patterns such as the periodic responses observed during NFT browsing. The optimal parameter combination is $W = 40$ and $s = 5$, with an accuracy of over 0.9170. We adopt these parameters for NFTracker.

F. Comparison With State-of-the-Art Methods

In this section, we evaluate the behavior identification ability of NFTracker in real-world NFT traffic.

Experimental Setting: To evaluate the performance of NFTracker in identifying NFT fine-grained behavioral traffic over the encrypted tunnel, we compare our proposed NFTracker with prior DApp traffic classification and behavioral traffic identification methods on the D_2 dataset, including GraphDApp [8], ActiveTracker [10], PACKETPRINT [32], Hou et al. [30], and BehavSniffer [31], as well as single CNN (NFTracker-C) or Transformer (NFTracker-T) variants of NFTracker as baselines.

Result: As shown in Table IV, NFTracker achieves superior performance with an average precision of 0.9245, recall of 0.9179, and F1-score of 0.9212, outperforming all baseline methods. The hybrid CNN-Transformer architecture demonstrates significant advantages over its individual components. PACKETPRINT [32] exhibits the closest performance, while BehavSniffer [31] lags due to its limited capability in distinguishing behaviors with overlapping traffic patterns.

The confusion matrix analysis in Figure 11 further elucidates class-specific performance. NFT browsing achieves 0.9984 accuracy, attributed to its strong periodicity of user requests and platform responses, minimal wallet interactions, and a response phase dominated by continuous MTU size data packets, which form a more obvious traffic characteristic. Minting exhibits partial misclassification with selling due to shared wallet authorization processes. Notably, purchasing and selling show the highest mutual misclassification rates. It is caused by their nearly identical interaction logic, including wallet verification and payment phases.

These results validate NFTracker's robustness against feature homogeneity and encryption-induced obfuscation. The spatio-temporal feature extraction effectively isolates localized action fingerprints, while the hybrid model synergizes spatial patterns and temporal dependencies. In contrast, methods relying on static statistical features [30] or graph structures [8] fail to capture dynamic traffic variations, leading to degraded performance. Experimental results demonstrate the NFTracker's practical efficacy in encrypted tunnel scenarios.

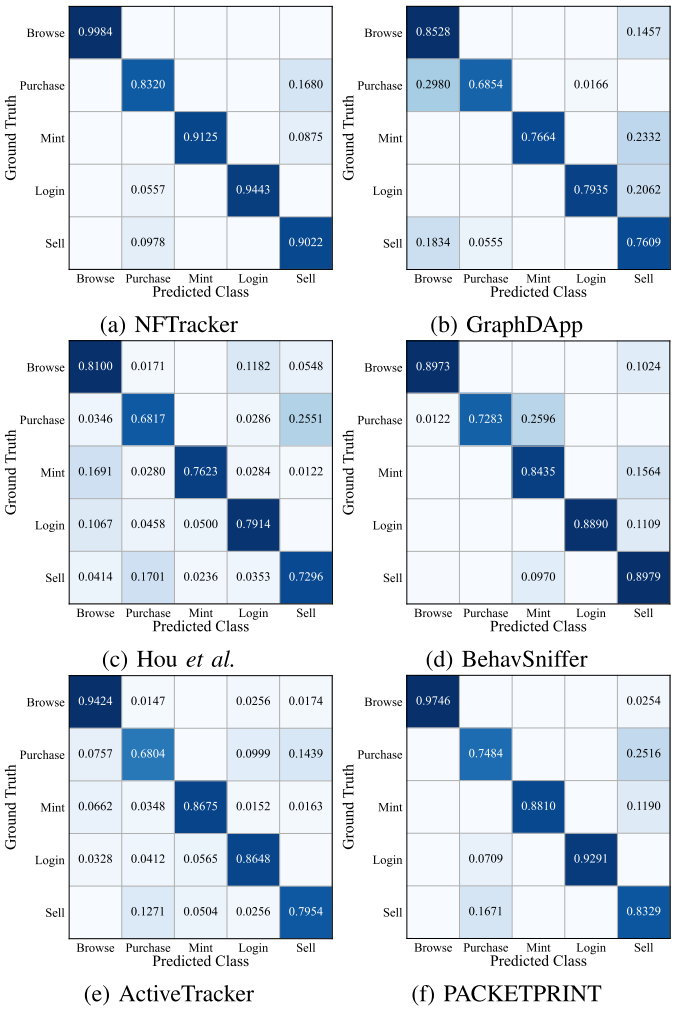


Fig. 11. Confusion matrices of each method.

VI. DISCUSSION

In this section, we discuss the limitations of our method and potential directions for future work.

A. Real-World Applicability

Composite user behaviors present challenges for existing approaches [30], [31], such as browsing and completing multiple wallet authorizations during NFT minting, or initiating combined transactions after continuous browsing of multiple collectibles. NFTracker addresses these through burst-WD segmentation to isolate behaviors, sliding-window-based feature extraction to capture action patterns, and a hybrid architecture for local and global dependencies. For instance, in sequential "browse-login-minting-sell" workflows, NFTracker could effectively capture stage-specific features and learn behavioral logic through these features. Additionally, mainstream Ethereum Layer 2 rollups already support NFTs, and their off-chain execution with batched settlement to Layer 1 can alter the timing, ordering, and spacing of packets, which could induce distinct features in encrypted traffic. NFTracker is layer-agnostic because it operates on packet-level features, though specific Layer 2 environments may require minor retraining. In future work, we will further improve accuracy for practical deployment.

B. Defensive Countermeasures Against NFTracker

NFTracker achieves fine-grained identification by leveraging the spatio-temporal features of NFT behavioral traffic in encrypted tunnels. To counter such mechanisms, defenders could design traffic obfuscation strategies to interfere with its critical feature extraction process. A viable solution is for defenders to first collect behavioral traffic from target NFT platforms, analyze spatio-temporal distribution patterns, and employ generative adversarial networks to generate obfuscated traffic based on real traffic characteristics. They could constrain the WD between generated and real traffic within local time windows, while globally disrupting directional correlations in packet length distributions. It causes NFTracker's burst segmentation algorithm to produce false boundaries, invalidating its spatio-temporal feature extraction. The design and effectiveness validation of such adversarial obfuscation mechanisms represent an interesting future research direction.

C. Multiplexing

It combines multiple logical streams over a single TCP connection, thereby improving network transmission efficiency. Like V2Ray's Mux.Cool protocol [35], which embeds application-layer protocol headers with subflow IDs and length identifiers within encrypted TLS streams. Its multiplexed information is encrypted with application data, appearing as ordinary TLS traffic and difficult to identify by intermediate devices. It results in an IP packet containing multiple application-layer messages, simultaneously affecting the packet size and time gap characteristics. However, currently, due to Mux.cool's inability to significantly optimize latency, its usage scope is limited. The existing method [32] uses packet size filtering to alleviate the noise caused by multiplexing to a certain extent. However, many behavioral data packets will be lost after screening, resulting in an insufficient representation of behavioral features. We do not yet have an effective solution for behavior identification under multiplexing. We will leave this to our future work.

D. Packet Padding

It is a traffic obfuscation technique designed to conceal packet length characteristics by appending random padding data to packets [36], enhancing anti-detection capabilities. NFTracker relies on timing gaps and packet sequences to characterize behaviors. While packet padding introduces noise by invalidating packet length features, it does not affect packet direction or timing gaps. Consequently, packet padding may cause minor degradation in NFTracker's identification performance but would not fundamentally disrupt its functionality. Additionally, the extra network overhead induced by packet padding could lead to significant latency, degrading the user experience.

VII. Conclusion

We present a novel framework named NFTracker for fine-grained NFT behavioral traffic identification over encrypted tunnels. NFTracker addresses the core challenges of inexact segmentation of continuous behaviors and the homogeneity of NFT traffic features in encrypted traffic by integrating burst-WD-based segmentation, sliding window feature extraction, and a hybrid CNN-Transformer model for classification. Specifically, NFTracker synergistically captures both localized spatial patterns and global temporal dependencies, achieving precise classification of five NFT behaviors, including browsing, wallet login, purchasing, selling, and minting. Our experimental study evaluates the performance of traffic segmentation and behavior identification of NFTracker. The results demonstrate that NFTracker provides a practical and effective solution for identifying fine-grained NFT behavioral traffic in encrypted tunnel scenarios.

References

[1] S. G. Almeda and B. Hartmann, "NFT art world: The influence of decentralized systems on the development of novel online creative communities and cooperative practices," in *Proc. ACM Designing Interact. Syst. Conf.*, Jul. 2023, pp. 353–370.

[2] H. Chen, Z. Yang, and T. Lyu, "Empirical investigation of digital collectibles purchase intention: The roles of value, risks, identification, and scarcity," *Int. J. Hum.–Comput. Interact.*, vol. 41, no. 16, pp. 9861–9880, Nov. 2024.

[3] Y.-J. Yang and J.-L. Wang, "Non-fungible token (NFT) games: A literature review," in *Proc. Int. Conf. Cyber Manage. Eng. (CyMaEn)*, Jan. 2023, pp. 251–254.

[4] Y. An, Y. He, F. R. Yu, J. Li, J. Chen, and V. C. M. Leung, "An HTTP anomaly detection architecture based on the Internet of Intelligence," *IEEE Trans. Cognit. Commun. Netw.*, vol. 8, no. 3, pp. 1552–1565, Sep. 2022.

[5] S. S. Roy, D. Das, P. Bose, C. Kruegel, G. Vigna, and S. Nilizadeh, "Unveiling the risks of NFT promotion scams," in *Proc. Int. AAAI Conf. Web Soc. Media (ICWSM)*, 2023, pp. 1367–1380.

[6] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surveys*, vol. 54, no. 6, pp. 1–35, Jul. 2021.

[7] M. Liu et al., "Enhanced detection of obfuscated HTTPS tunnel traffic using heterogeneous information network," *Comput. Netw.*, vol. 257, Feb. 2025, Art. no. 110975.

[8] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[9] I. Karunanayake, J. Jiang, N. Ahmed, and S. K. Jha, "Exploring uncharted waters of website fingerprinting," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1840–1854, 2024.

[10] D. Li, W. Li, X. Wang, C.-T. Nguyen, and S. Lu, "App trajectory recognition over encrypted internet traffic based on deep neural network," *Comput. Netw.*, vol. 179, Oct. 2020, Art. no. 107372.

[11] X. Hu, Z. Shu, Z. Tong, G. Cheng, R. Li, and H. Wu, "Fine-grained Ethereum behavior identification via encrypted traffic analysis with serialized backward inference," *Comput. Netw.*, vol. 237, Dec. 2023, Art. no. 110110.

[12] *V2ray*. Accessed: Apr. 30, 2025. [Online]. Available: https://www.v2ray.com

[13] *VMess*. Accessed: Apr. 30, 2025. [Online]. Available: https://www.v2fly.org/en_US/developer/protocols/vmess.html

[14] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[15] W. Entriken, D. Shirley, J. Evans, and N. Sachs. (2018). *ERC-721: Non-Fungible Token Standard*. Accessed: Apr. 30, 2025. [Online]. Available: https://eips.ethereum.org/EIPS/eip-721

[16] W. Radomski, A. Cooke, P. Castonguay, J. Therien, E. Binet, and R. Sandford. (2018). *ERC-1155: Multi Token Standard*. Accessed: Apr. 30, 2025. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1155

[17] T. Wang et al., "COMET: NFT price prediction with wallet profiling," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, New York, NY, USA, Aug. 2024, pp. 5893–5904.

[18] Y. Niu, X. Li, H. Peng, and W. Li, "Unveiling wash trading in popular NFT markets," in *Companion Proc. ACM Web Conf.*, New York, NY, USA, May 2024, pp. 730–733.

[19] J. Huang et al., "Unveiling the paradox of NFT prosperity," in *Proc. ACM Web Conf.*, New York, NY, USA, May 2024, pp. 167–177.
[20] Y. Cao et al., "NFTracer: Tracing NFT impact dynamics in transaction-flow substitutive systems with visual analytics," *IEEE Trans. Vis. Comput. Graphics*, vol. 31, no. 8, pp. 4369–4386, Aug. 2025.
[21] Z. Che et al., "Across-platform detection of malicious cryptocurrency accounts via interaction feature learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4783–4798, 2025.
[22] D. Xue, M. Kallitsis, A. Houmansadr, and R. Ensafi, "Fingerprinting obfuscated proxy traffic with encapsulated tls handshakes," in *Proc. USENIX Secur. Symp.*, 2024, pp. 2689–2706.
[23] D. Xue et al., "OpenVPN is open to VPN fingerprinting," *Commun. ACM*, vol. 68, no. 1, pp. 79–87, Jan. 2025.
[24] S. Chen, S. Chen, H. He, X. Jiang, J. Yang, and S. Cheng, "Causality correlation and context learning aided robust lightweight multi-tab website fingerprinting over encrypted tunnel," in *Proc. IEEE Conf. Comput. Commun.*, May 2024, pp. 761–770.
[25] X. Ma et al., "Website fingerprinting on encrypted proxies: A flow-context-aware approach and countermeasures," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 1904–1919, Jun. 2024.
[26] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, and L. Zhu, "Robust detection of malicious encrypted traffic via contrastive learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4228–4242, 2025.
[27] S.-J. Xu, K.-C. Kong, X.-B. Jin, and G.-G. Geng, "Unveiling traffic paths: Explainable path signature feature-based encrypted traffic classification," *Comput. Secur.*, vol. 150, Mar. 2025, Art. no. 104283.
[28] F. Aiolli, M. Conti, A. Gangwal, and M. Polato, "Mind your wallet's privacy: Identifying Bitcoin wallet apps and user's actions through network traffic analysis," in *Proc. 34th ACM/SIGAPP Symp. Appl. Comput.*, Apr. 2019, pp. 1484–1491.
[29] D. Ahmed, A. Sabir, and A. Das, "Spying through your voice assistants: Realistic voice command fingerprinting," in *Proc. USENIX Secur. Symp.*, Aug. 2023, pp. 2419–2436.
[30] C. Hou, J. Shi, C. Kang, Z. Cao, and X. Gang, "Classifying user activities in the encrypted WeChat traffic," in *Proc. IEEE 37th Int. Perform. Comput. Commun. Conf. (IPCCC)*, Nov. 2018, pp. 1–8.
[31] T. Wu et al., "BehavSniffer: Sniff user behaviors from the encrypted traffic by traffic burst graphs," in *Proc. 20th Annu. IEEE Int. Conf. Sens., Commun., Netw. (SECON)*, Sep. 2023, pp. 456–464.
[32] J. Li et al., "Robust app fingerprinting over the air," *IEEE/ACM Trans. Netw.*, vol. 32, no. 6, pp. 5065–5080, Dec. 2024.
[33] B. Gao, W. Liu, G. Liu, and F. Nie, "Resource knowledge-driven heterogeneous graph learning for website fingerprinting," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 3, pp. 968–981, Jun. 2024.
[34] F. Wu et al., "Multi-variate time series prediction of traffic and users for dynamic RRH-BBU mapping in C-RAN," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10557–10572, Oct. 2025.
[35] *Mux.Cool Protocol*. Accessed: Apr. 30, 2025. [Online]. Available: https://v2ray.com/developer/protocols/muxcool.html
[36] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 607–624.

Ke Ding (Graduate Student Member, IEEE) received the M.S. degree in software engineering from East China Normal University in 2023. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Southeast University, Nanjing, China. His research interests include blockchain security and network security.

Xiaoyan Hu (Member, IEEE) received the Ph.D. degree in computer architecture from Southeast University, Nanjing, China, in 2015. She visited the NetSec Laboratory, Colorado State University, from 2010 to 2012. She is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University. Her research interests include network intrusion detection, network security, and future network architecture.

Zhuozhuo Shu received the B.S. and M.S. degrees from the School of Cyber Science and Engineering, Southeast University, Nanjing, China, in 2021 and 2024, respectively. His research interests include network traffic analysis.

Guang Cheng (Member, IEEE) received the Ph.D. degree in computer science from Southeast University, Nanjing, China, in 2003. From 2006 to 2007, he held a post-doctoral position with the School of Electrical and Computer Engineering, Georgia Institute of Technology. He is currently a Full Professor with the School of Cyber Science and Engineering, Southeast University. His research interests include network security, network measurement, and encrypted traffic analysis.

Ruidong Li (Senior Member, IEEE) received the B.S. degree in engineering from Zhejiang University, Hangzhou, China, in 2001, and the M.S. and Ph.D. degrees from the University of Tsukuba, Japan, in 2005 and 2008, respectively. He was a Researcher with the Network Architecture Laboratory, National Institute of Information and Communications Technology (NICT). He is currently an Associate Professor with the Institute of Science and Engineering, Kanazawa University. His current research interests include the Internet of Things, future networks, information-centric networking, security/secure architectures of future networks, and next-generation wireless networking. He serves on the editorial board for IEEE INTERNET OF THINGS JOURNAL and *KSII Transactions on Internet and Information Systems*.

Hua Wu (Member, IEEE) received the Ph.D. degree in computer application technology from Southeast University, Nanjing, China, in 2010. She is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University. Her research interests include computer network security and encrypted traffic analysis.